

Lecture 6: Application Security I

Security

Francesco Bariatti

18 March 2026



Universiteit
Leiden
The Netherlands



Software vulnerabilities

- At any time, many programs run on **shared and/or public** machines
 - e.g. web servers, databases, production equipment, ...
- **Exploiting** vulnerabilities in a **program** can allow attackers to get access to the machines
- Vulnerabilities can stem from programming bugs, or programming languages unwieldy behaviour
 - e.g. in C, programmer is responsible for proper *memory management*

1. Buffer overflow
 - Heap overflow

2. Integer overflow

3. String formatting vulnerabilities

4. Other vulnerabilities

Buffer overflow: ever relevant

2025 CWE Top 10 KEV Weaknesses

[Top 25 Home](#)
[Share via: !\[\]\(e78f798d4ea5c530c9db49e7d26e6b95_img.jpg\)](#)
[View in table format](#)
[KEV Key Insights](#)
[KEV Methodology](#)

1

Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

[CWE-78](#) | CVEs in KEV: 20 | Rank Last Year: 3 (up 2) ▲

2

Use After Free

[CWE-416](#) | CVEs in KEV: 14 | Rank Last Year: 9 (up 7) ▲

3

Out-of-bounds Write

[CWE-787](#) | CVEs in KEV: 12 | Rank Last Year: 1 (down 2) ▼

4

Missing Authentication for Critical Function

[CWE-306](#) | CVEs in KEV: 11 | Rank Last Year: 7 (up 3) ▲

5

Deserialization of Untrusted Data

[CWE-502](#) | CVEs in KEV: 11 | Rank Last Year: 5

6

Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

[CWE-22](#) | CVEs in KEV: 10 | Rank Last Year: 6

7

Improper Control of Generation of Code ('Code Injection')

[CWE-94](#) | CVEs in KEV: 7 | Rank Last Year: 4 (down 3) ▼

8

Authentication Bypass Using an Alternate Path or Channel

[CWE-288](#) | CVEs in KEV: 6 | Rank Last Year: 14 (up 6) ▲

9

Heap-based Buffer Overflow

[CWE-122](#) | CVEs in KEV: 6 | Rank Last Year: 11 (up 2) ▲

10

Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

[CWE-79](#) | CVEs in KEV: 7 | Rank Last Year: 21 (up 11) ▲

https://cwe.mitre.org/top25/archive/2025/2025_kev_list.html

Buffer overflow: ever relevant

2025 CWE Top 10 KEV Weaknesses

[Top 25 Home](#)
[Share via:](#)
[View in table format](#)
[KEV Key Insights](#)
[KEV Methodology](#)

1

Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

[CWE-78](#) | CVEs in KEV: 20 | Rank Last Year: 3 (up 2) ▲

2

Use After Free

[CWE-416](#) | CVEs in KEV: 14 | Rank Last Year: 9 (up 7) ▲

3

Out-of-bounds Write

[CWE-787](#) | CVEs in KEV: 12 | Rank Last Year: 1 (down 2) ▼

4

Missing Authentication for Critical Function

[CWE-306](#) | CVEs in KEV: 11 | Rank Last Year: 7 (up 3) ▲

5

Deserialization of Untrusted Data

[CWE-502](#) | CVEs in KEV: 11 | Rank Last Year: 5

6

Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

[CWE-22](#) | CVEs in KEV: 10 | Rank Last Year: 6

7

Improper Control of Generation of Code ('Code Injection')

[CWE-94](#) | CVEs in KEV: 7 | Rank Last Year: 4 (down 3) ▼

8

Authentication Bypass Using an Alternate Path or Channel

[CWE-288](#) | CVEs in KEV: 6 | Rank Last Year: 14 (up 6) ▲

9

Heap-based Buffer Overflow

[CWE-122](#) | CVEs in KEV: 6 | Rank Last Year: 11 (up 2) ▲

10

Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

[CWE-79](#) | CVEs in KEV: 7 | Rank Last Year: 21 (up 11) ▲

https://cwe.mitre.org/top25/archive/2025/2025_kev_list.html

What is a buffer?

What is a buffer?

A **buffer** is a **(contiguous) memory region** associated with a variable.

A **buffer overflow** occurs when the program (is tricked to) access the memory **beyond the theoretical limits of a buffer**.

```
>>> l = [1,2,3]
>>> l[42] = 67
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list assignment index out of range
Python preventing you from overflowing a buffer
```

- C standard states that accessing memory beyond a variable's limit is *undefined behaviour*
⇒ Compiler allowed to assume that it will never happen
- Protection against overflow lies with OS, compiler, and programmer
- OS has protections and compilers have checks to prevent overflow
 - But many checks just raise a warning and are ignored
 - Some situations impossible to predict at compile time ($\approx$ halting problem)
 - Runtime boundary checks can be costly, especially in low-latency scenarios

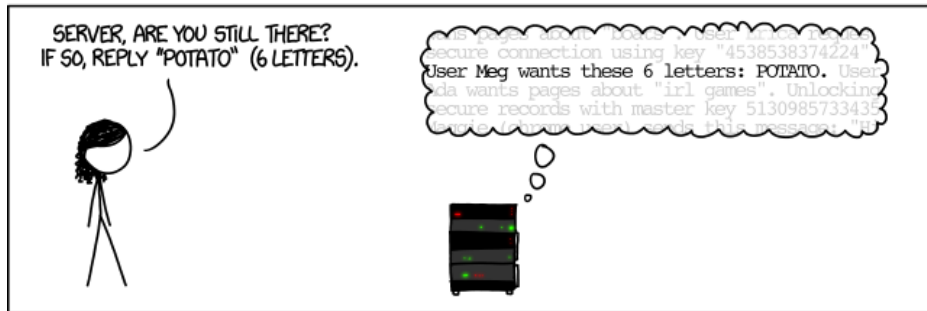
Read overflow – The Heartbleed vulnerability

- <https://nvd.nist.gov/vuln/detail/CVE-2014-0160>
- Vulnerability in the OpenSSL library
 - Most common library for secure communication and cryptography
- Vulnerability in the “heartbeat” protocol allows to read memory of cryptographic library
 - e.g. cryptographic keys and passwords

⇒ High risk of enabling further attacks

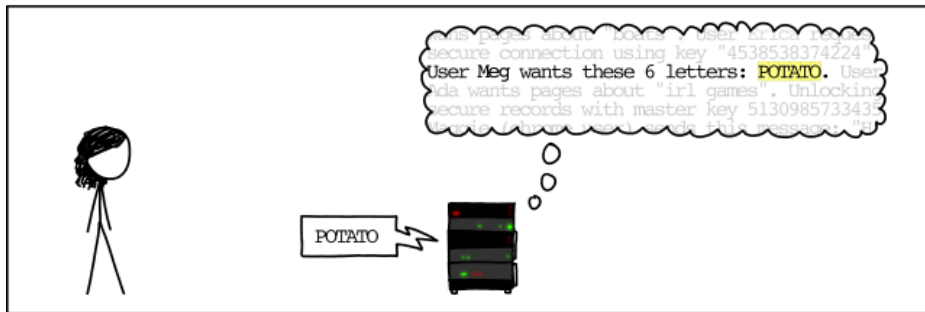
Read overflow – The Heartbleed vulnerability

HOW THE HEARTBLEED BUG WORKS:



<https://xkcd.com/1354/>

Read overflow – The Heartbleed vulnerability



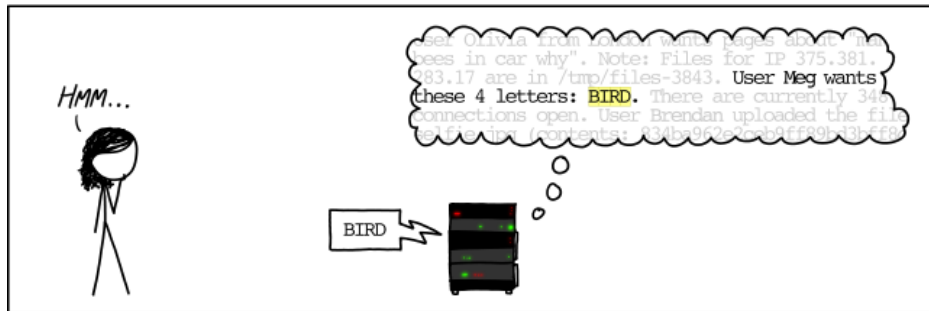
<https://xkcd.com/1354/>

Read overflow – The Heartbleed vulnerability



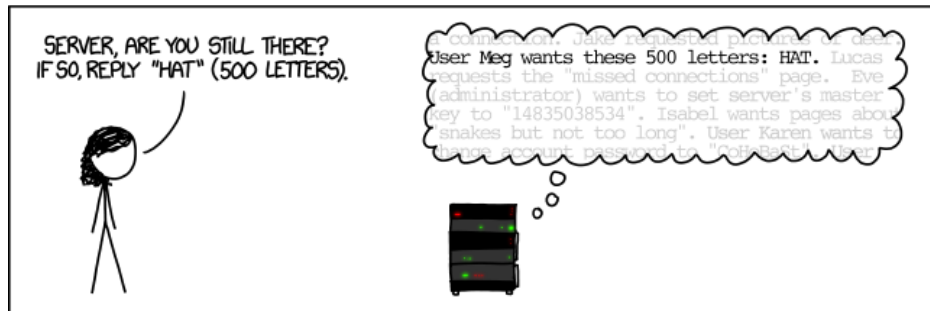
<https://xkcd.com/1354/>

Read overflow – The Heartbleed vulnerability



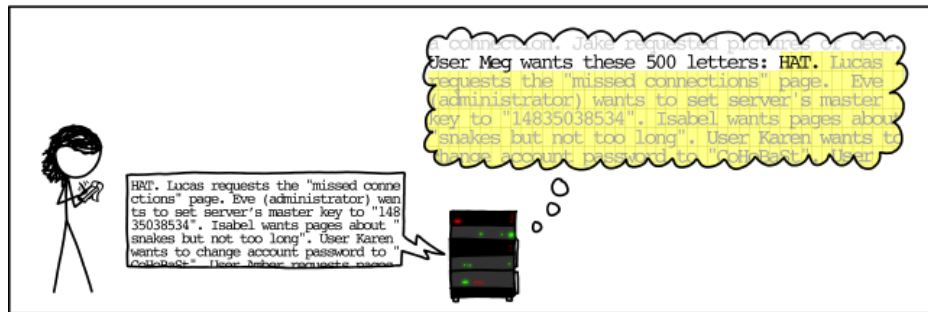
<https://xkcd.com/1354/>

Read overflow – The Heartbleed vulnerability



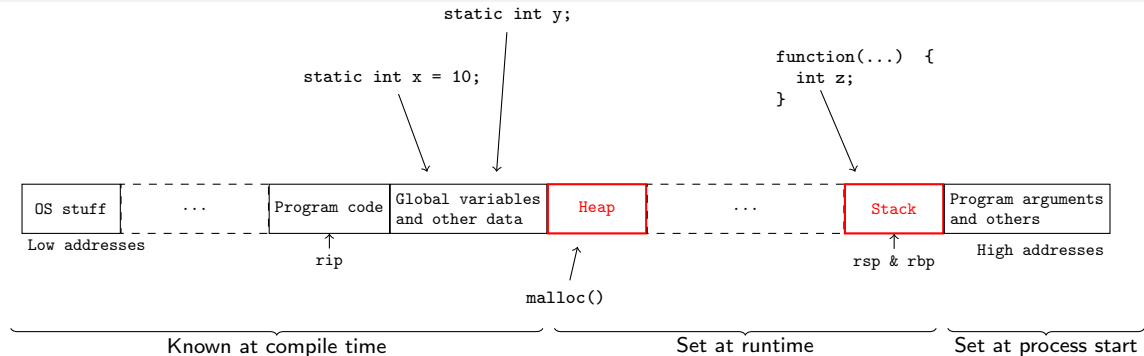
<https://xkcd.com/1354/>

Read overflow – The Heartbleed vulnerability



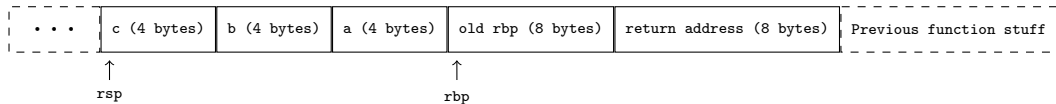
<https://xkcd.com/1354/>

Reminder: program memory layout



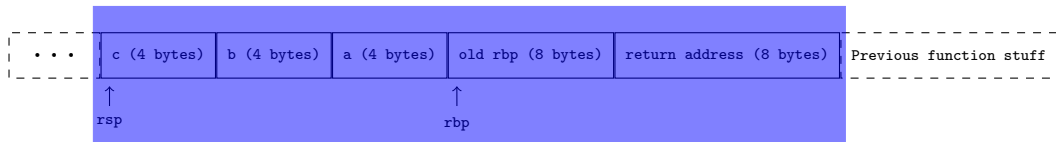
Inside a function: stack memory

```
int add(int a, int b) {  
    int c;  
    c = a + b;  
    return c;  
}
```



Inside a function: stack memory

```
int add(int a, int b) {  
    int c;  
    c = a + b;  
    return c;  
}
```



Stack frame of `add`

Our first buffer (not) overflow

```

1  int f(char* arg1) {
2      char buffer[4];
3      strcpy(buffer, arg1);
4      // Rest of the function
5  }
```

```

6  void main() {
7      char* myStr = "Hi!"
8      f(myStr);
9      // Rest of the function
10 }
```



rip: "line 8"
...

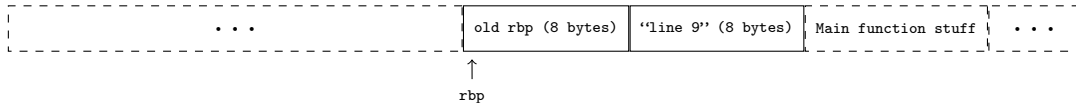
Our first buffer (not) overflow

```

1  int f(char* arg1) {
2      char buffer[4];
3      strcpy(buffer, arg1);
4      // Rest of the function
5  }
```

```

6  void main() {
7      char* myStr = "Hi!"
8      f(myStr);
9      // Rest of the function
10 }
```



```

rip: "line 2"
...
```

Our first buffer (not) overflow

```

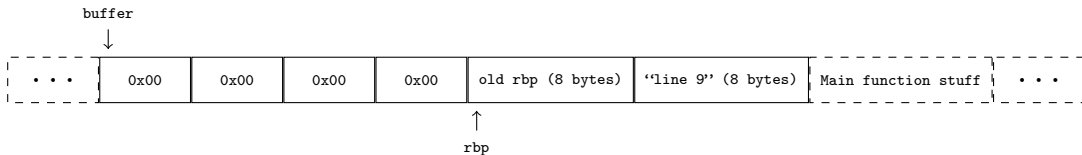
1  int f(char* arg1) {
2      char buffer[4];
3      strcpy(buffer, arg1);
4      // Rest of the function
5  }

```

```

6  void main() {
7      char* myStr = "Hi!"
8      f(myStr);
9      // Rest of the function
10 }

```



```

rip: "line 3"
...

```

Our first buffer (not) overflow

```

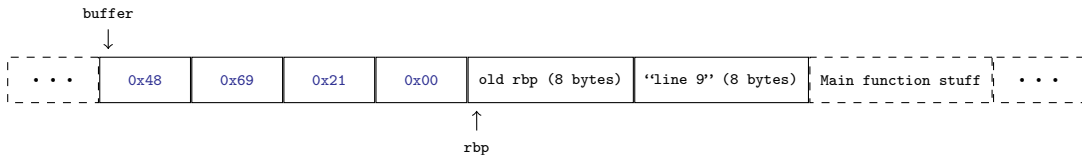
1  int f(char* arg1) {
2      char buffer[4];
3      strcpy(buffer, arg1);
4      // Rest of the function
5  }

```

```

6  void main() {
7      char* myStr = "Hi!"
8      f(myStr);
9      // Rest of the function
10 }

```



```

rip: "line 4"
...

```

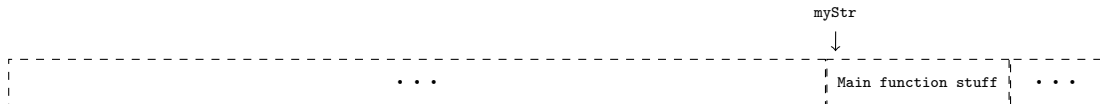
Our first buffer overflow

```

int f(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    // Rest of the function
}

6 void main() {
7     char* myStr = "I have hacked you!!";
8     f(myStr);
9     // Rest of the function
10 }

```

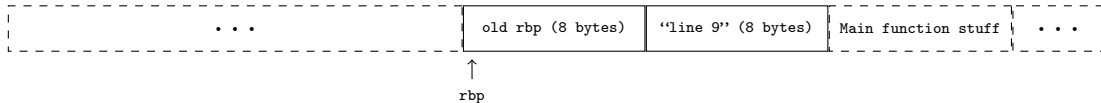


rip: "line 8"
...

Our first buffer overflow

```
int f(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    // Rest of the function
}
```

```
6 void main() {
7     char* myStr = "I have hacked you!!";
8     f(myStr);
9     // Rest of the function
10 }
```

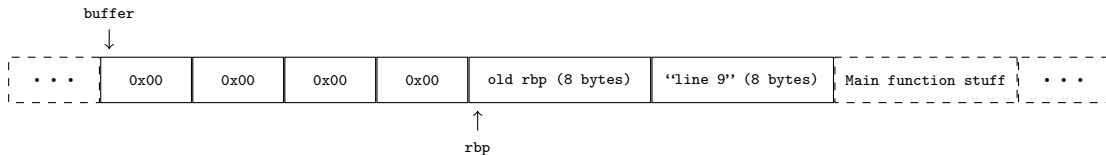


```
rip: "line 2"
...
```


Our first buffer overflow

```
int f(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    // Rest of the function
}
```

```
6 void main() {
7     char* myStr = "I have hacked you!!";
8     f(myStr);
9     // Rest of the function
10 }
```

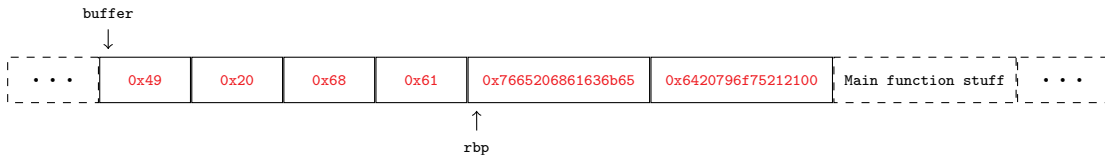


```
rip: "line 3"
...
```

Our first buffer overflow

```
int f(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    // Rest of the function
}
```

```
6 void main() {
7     char* myStr = "I have hacked you!!";
8     f(myStr);
9     // Rest of the function
10 }
```



rip: "line 4"
...

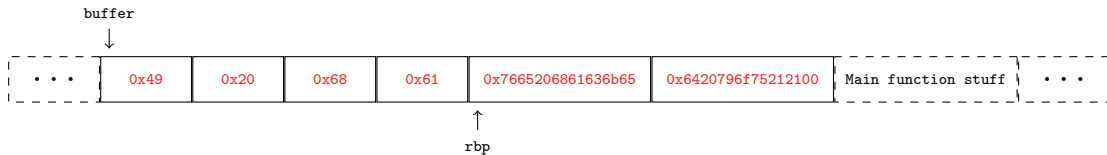
Our first buffer overflow

```

int f(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    // Rest of the function
}

6 void main() {
7     char* myStr = "I have hacked you!!";
8     f(myStr);
9     // Rest of the function
10 }

```



Upon return from f:

- rbp will contain 0x7665206861636b65 instead of the original value
 - Highly likely to crash the program if used → might need to guess a better value
- Program will execute instructions at address 0x6420796f75212100 instead of returning to main

Buffer overflow payload and challenges

Now that we can overwrite program memory, what should we write?

Buffer overflow payload and challenges

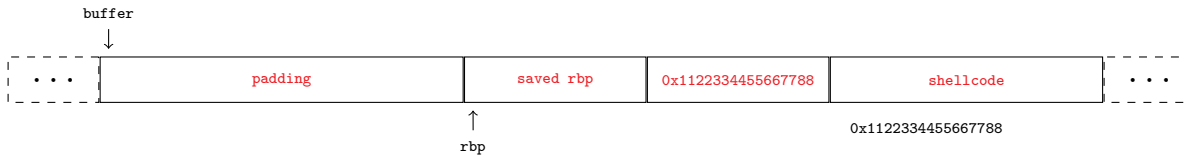
Now that we can overwrite program memory, what should we write?

```
1  mov al, 60
2  xor rdi, rdi
3  syscall
```

```
1  0xb0 0x3c
2  0x48 0x31 0xff
3  0x0f 0x05
```

- Remember: instructions are just bytes!
- Goal: injecting instructions and jumping to them
 - **Shellcode**: binary instructions for running a shell → see `execve` syscall
- ☹ Binary depends on machine architecture
- ☹ Code needs to be self-contained (can not load libraries)
- ☹ Code can not contain all-zero bytes
 - Why?
- 👍 Most machines have the same architecture and use the same environments

Guessing the right address



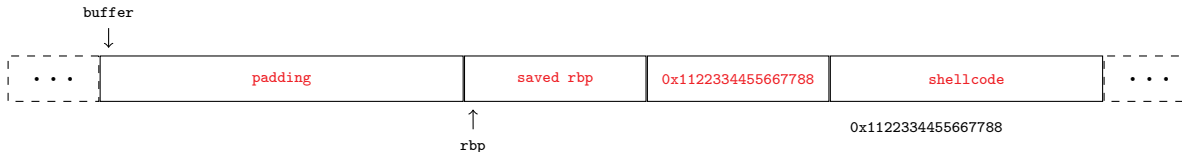
Guessing the right address



- How to make sure that we pick the right address?

⚠ Missing by one byte will completely misinterpret the code

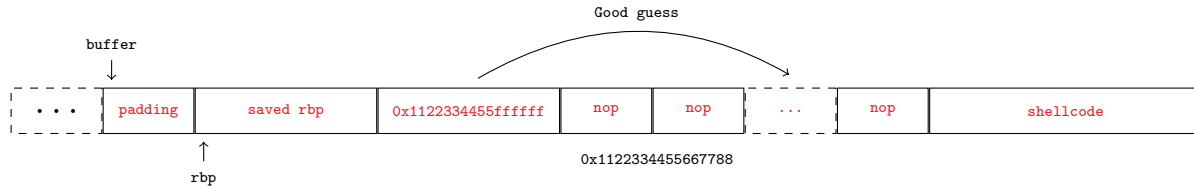
Guessing the right address



- How to make sure that we pick the right address?
 - ⚠ Missing by one byte will completely misinterpret the code
- On some (old or limited) systems, stack position is fixed and known
 - Stack always starts from same fixed address
 - Stack grows but not very deeply

NOP sled

- nop or “no-op(eration)” is a one-byte instruction (0x90 on x86-64) that does nothing
- If we have a bunch of those instructions, executing any of those bytes will just move to the next ...
- ... eventually arriving at the actual payload



Overflow exploits without custom code

- Possible to exploit a buffer overflow without writing our own (shell)code

```
1  int f(char* arg1) {  
2      int authenticated = 0;  
3      char buffer[4];  
4      strcpy(buffer, arg1);  
5      if (authenticated) {  
6          // Restricted area  
7      }  
8  }
```

```
9  void main() {  
10      char* myStr = "BBBBBBBB";  
11      f(myStr);  
12      // Rest of the function  
13  }
```

Overflow exploits without custom code

- Possible to exploit a buffer overflow without writing our own (shell)code

```

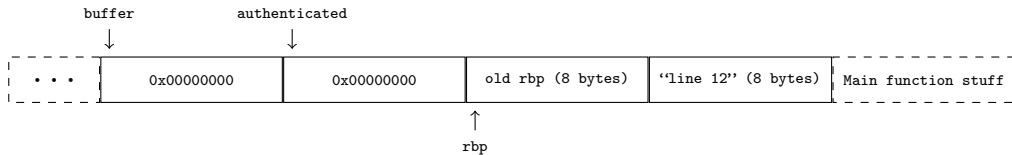
1  int f(char* arg1) {
2      int authenticated = 0;
3      char buffer[4];
4      strcpy(buffer, arg1);
5      if (authenticated) {
6          // Restricted area
7      }
8  }

```

```

9  void main() {
10     char* myStr = "BBBBBBBB";
11     f(myStr);
12     // Rest of the function
13 }

```



Overflow exploits without custom code

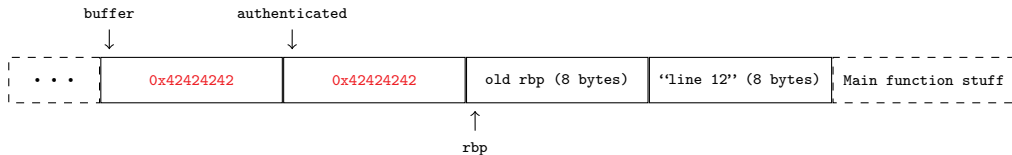
- Possible to exploit a buffer overflow without writing our own (shell)code

```

1  int f(char* arg1) {
2      int authenticated = 0;
3      char buffer[4];
4      strcpy(buffer, arg1);
5      if (authenticated) {
6          // Restricted area
7      }
8  }
```

```

9  void main() {
10     char* myStr = "BBBBBBBB";
11     f(myStr);
12     // Rest of the function
13 }
```



- Reminder: any non-0 value is considered "true" in C

Heap overflow

- Until now, we have been doing “**stack smashing**”
- ... but programs use another memory region: the **heap**!
 - Memory reserved via `malloc`
 - Memory used to survive beyond a function's end
 - Many high-level languages (Java, Python) use the heap a lot for object-oriented programming

Heap overflow

- Until now, we have been doing “**stack smashing**”
- ... but programs use another memory region: the **heap**!
 - Memory reserved via `malloc`
 - Memory used to survive beyond a function's end
 - Many high-level languages (Java, Python) use the heap a lot for object-oriented programming

```
void f() {  
    char* buf = (char*) malloc(4);  
    char* buf2 = (char*) malloc(42);  
    char* user_input = /*read user input*/  
    strcpy(buf, user_input);  
    // Rest of the function  
}
```

Heap overflow

- Until now, we have been doing “**stack smashing**”
- ... but programs use another memory region: the **heap**!
 - Memory reserved via `malloc`
 - Memory used to survive beyond a function's end
 - Many high-level languages (Java, Python) use the heap a lot for object-oriented programming

```
void f() {  
    char* buf = (char*) malloc(4);  
    char* buf2 = (char*) malloc(42);  
    char* user_input = /*read user input*/  
    strcpy(buf, user_input);  
    // Rest of the function  
}
```

- If user input > 4, then overwrite heap
- It might be `buf2`, it might be something unrelated!

Heap overflow consequences

- Overflow into adjacent objects
- In object-oriented programming: overflow into vtables (class information)
 - ⇒ Change object methods!
- Overflow heap metadata itself

1. Buffer overflow
 - Heap overflow
2. Integer overflow
3. String formatting vulnerabilities
4. Other vulnerabilities

Integer overflow

```
1 char** read_network_packet() {  
2     packet* p = wait_for_packet();  
3     int n_strings = p->header->n_strings;  
4     char** strings = (char**) malloc(n_strings * sizeof(char*));  
5     for (int i = 0; i < n_strings; i++)  
6         strings[i] = p->get_string(i);  
7     return strings;  
8 }
```

What is the problem?

Integer overflow

```
1 char** read_network_packet() {  
2     packet* p = wait_for_packet();  
3     int n_strings = p->header->n_strings;  
4     char** strings = (char**) malloc(n_strings * sizeof(char*));  
5     for (int i = 0; i < n_strings; i++)  
6         strings[i] = p->get_string(i);  
7     return strings;  
8 }
```

What is the problem?

- `n_string` received over the network → unsanitized user input
- What if `n_string = 536870912`?
 - $536870912 * \text{sizeof}(\text{char}*) = 536870912 * 8 = 4294967296$
 - More than the max value in an int!
- $536870912 * \text{sizeof}(\text{char}*)$ **overflows to 0**
- `malloc` allocates 0 bytes: **all writes overflow**

Integer underflow

```
1 void process_input(const char *input, unsigned int length) {  
2     char buffer[100];  
3     // We only have space for 99 characters plus null terminator  
4     if (length - 1 > 100) {  
5         printf("Input too long!\n");  
6         return;  
7     }  
8     // Rest of function  
9 }
```

What is the problem?

Integer underflow

```
1 void process_input(const char *input, unsigned int length) {  
2     char buffer[100];  
3     // We only have space for 99 characters plus null terminator  
4     if (length - 1 > 100) {  
5         printf("Input too long!\n");  
6         return;  
7     }  
8     // Rest of function  
9 }
```

What is the problem?

- If $\text{length} = 0$, then $\text{length} - 1 = \text{MAX_INT}$
- **Will pass check** while conceptually being wrong

Buffer overflow summary

A **buffer** is a **(contiguous) memory region** associated with a variable.

A **buffer overflow** occurs when the program (is tricked to) access the memory **beyond the theoretical limits of a buffer**.

Dangers:

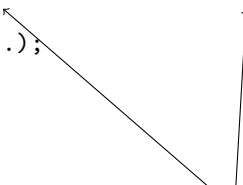
- Code injection
- Modify program's state
- Read restricted information

1. Buffer overflow
 - Heap overflow
2. Integer overflow
3. String formatting vulnerabilities
4. Other vulnerabilities

Formatting strings

Format string

```
1 printf("Printing a string: %s, and a number: %d\n", buffer, x);  
2 // Defined as  
3 printf(const char* format, ...);
```



Format specifiers

String formatting vulnerabilities

```
void safe() {  
    char buf[80];  
    // Assume this is safe  
    get_user_input(buf);  
    printf("User entered:\n",buf);  
    printf("%s",buf);  
}
```

```
1 void vulnerable() {  
2     char buf[80];  
3     // Assume this is safe  
4     get_user_input(buf);  
5     printf("User entered:\n",buf);  
6     printf(buf);  
7 }
```

What is the problem with the vulnerable function?

String formatting vulnerabilities

```
void safe() {  
    char buf[80];  
    // Assume this is safe  
    get_user_input(buf);  
    printf("User entered:\n", buf);  
    printf("%s", buf);  
}
```

```
1 void vulnerable() {  
2     char buf[80];  
3     // Assume this is safe  
4     get_user_input(buf);  
5     printf("User entered:\n", buf);  
6     printf(buf);  
7 }
```

What is the problem with the vulnerable function?

- **User can inject their own format**
- `printf` assumes that it is called with enough arguments
- It will happily print whatever happens to be where arguments normally are

printf attacks

- `printf("100% devilish");`

printf attacks

- `printf("100% devilish");` → Ignore spaces: prints the first “argument” as integer (%d)

printf attacks

- `printf("100% devilish");` → Ignore spaces: prints the first “argument” as integer (%d)
- `printf("%s");`

printf attacks

- `printf("100% devilish");` → Ignore spaces: prints the first “argument” as integer (%d)
- `printf("%s");` → Prints bytes at address given by first “argument”

printf attacks

- `printf("100% devilish");` → Ignore spaces: prints the first “argument” as integer (%d)
- `printf("%s");` → Prints bytes at address given by first “argument”
- `printf("100% no way!");`

printf attacks

- `printf("100% devilish");` → Ignore spaces: prints the first “argument” as integer (%d)
- `printf("%s");` → Prints bytes at address given by first “argument”
- `printf("100% no way!");` → **Writes** to address given by first “argument”

1. Buffer overflow
 - Heap overflow
2. Integer overflow
3. String formatting vulnerabilities
4. Other vulnerabilities

TOCTOU

```
1  if(can_access("file.txt") == 0) {  
2      printf("Unauthorized!\n");  
3      exit(1);  
4  }  
5  fd = open("file.txt", /*options*/);  
6  write(fd, /*content*/);
```

What is the problem?

TOCTOU

```
1  if(can_access("file.txt") == 0) {  
2      printf("Unauthorized!\n");  
3      exit(1);  
4  }  
5  fd = open("file.txt", /*options*/);  
6  write(fd, /*content*/);
```

What is the problem?

- **What if attacker changes meaning of file.txt on disk between line 2 and 6?**
 - e.g. symlink file.txt → /etc/passwd
- This is a **Time-Of-Check Time-Of-Use** vulnerability (TOCTOU)
 - Previous result of a check is used, but that result should have changed
 - A type of **race condition**

Supply-chain vulnerabilities

- Software uses many third-party libraries to make programming easier
- **Vulnerabilities in libraries become vulnerabilities in your program**
- Example: log4j vulnerability CVE-2021-44228
 - Widely-used logging framework
 - Allowed easy remote code execution on target machine
- **Can happen with any programming language**

<https://www.crowdstrike.com/en-gb/cybersecurity-101/exposure-management/log4j-vulnerability/>